CRM SALES DATA WAREHOUSE

Technical Project Documentation

Prepared by

Sabry Mahmoud Samy

Technology Stack

DuckDB | dbt Core | Apache Airflow | Python | SQL

1. Project Overview

The CRM Sales Data Warehouse is a modern ELT data pipeline built to ingest, transform, and model CRM sales data. The project utilizes DuckDB as a fast, embedded analytical database, dbt Core for robust SQL-based transformations, and Apache Airflow for workflow orchestration.

Source data consists of Accounts, Products, Sales Teams, and Sales Pipeline datasets. These are loaded directly into DuckDB, and then transformed through a structured dimensional modeling process to produce a star schema suitable for reporting and analytics.

2. System Architecture

The architecture follows a standard ELT (Extract, Load, Transform) pattern:

1. **Ingestion Layer (Python + DuckDB):** Python scripts extract raw CSV files and load them directly into the 'raw' schema in DuckDB.
2. **Staging Layer (dbt):** Views are created on top of the raw data to clean, cast, and standardize columns.
3. **Dimensional Layer (dbt):** Fact and dimension tables are materialized as physical tables representing the business entities and events.
4. **Orchestration (Airflow):** The entire pipeline (seeds, run, tests) is executed via a local Astro Airflow environment.

3. Data Modeling

The data model employs a classic Star Schema with one central fact table and four conforming dimensions. It is documented at three levels: conceptual, logical, and physical.

3.1 Conceptual Schema

The conceptual schema shows the business entities and their relationships: a central sales fact connected to the Sales Teams, Accounts, Products, and Date dimensions.

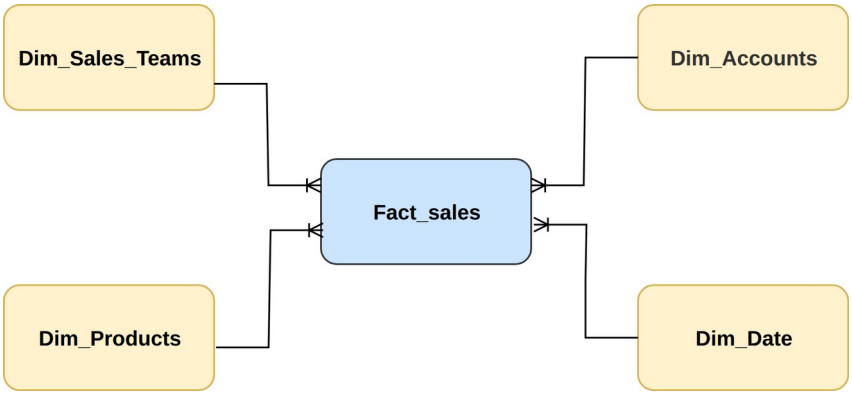


Figure 1: Conceptual schema of the CRM sales star schema.

3.2 Logical Schema

The logical schema defines the tables, keys, and attributes. Dim_Date is referenced twice by the fact table, through Engage_date_key and Close_date_key.

Table Name	Role	Primary Key
`dim_account`	Dimension	`account_key`
`dim_product`	Dimension	`product_key`
`dim_sales_agent`	Dimension	`sales_agent_key`
`dim_date`	Dimension	`date_key`
`fact_sales_opportunity`	Fact	`opportunity_key`

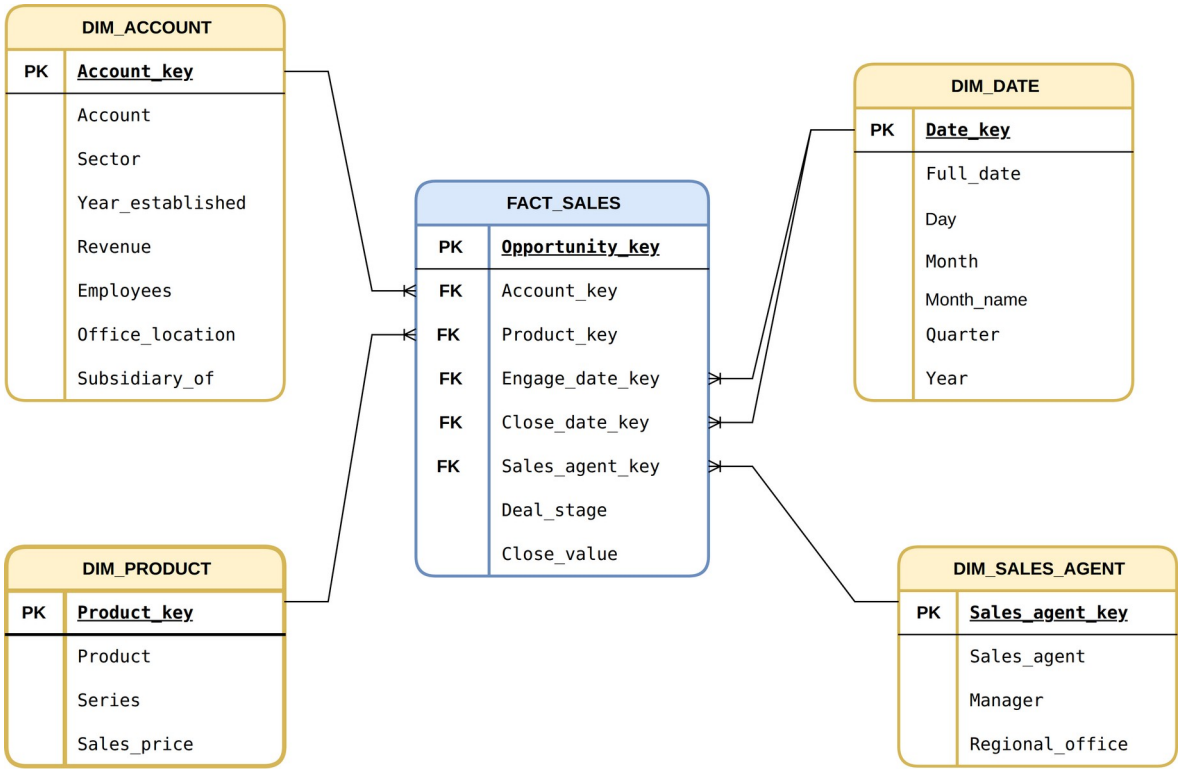


Figure 2: Logical schema showing primary keys (PK), foreign keys (FK), and attributes.

3.3 Physical Schema

The physical schema adds the column data types used when the tables are materialized in DuckDB.

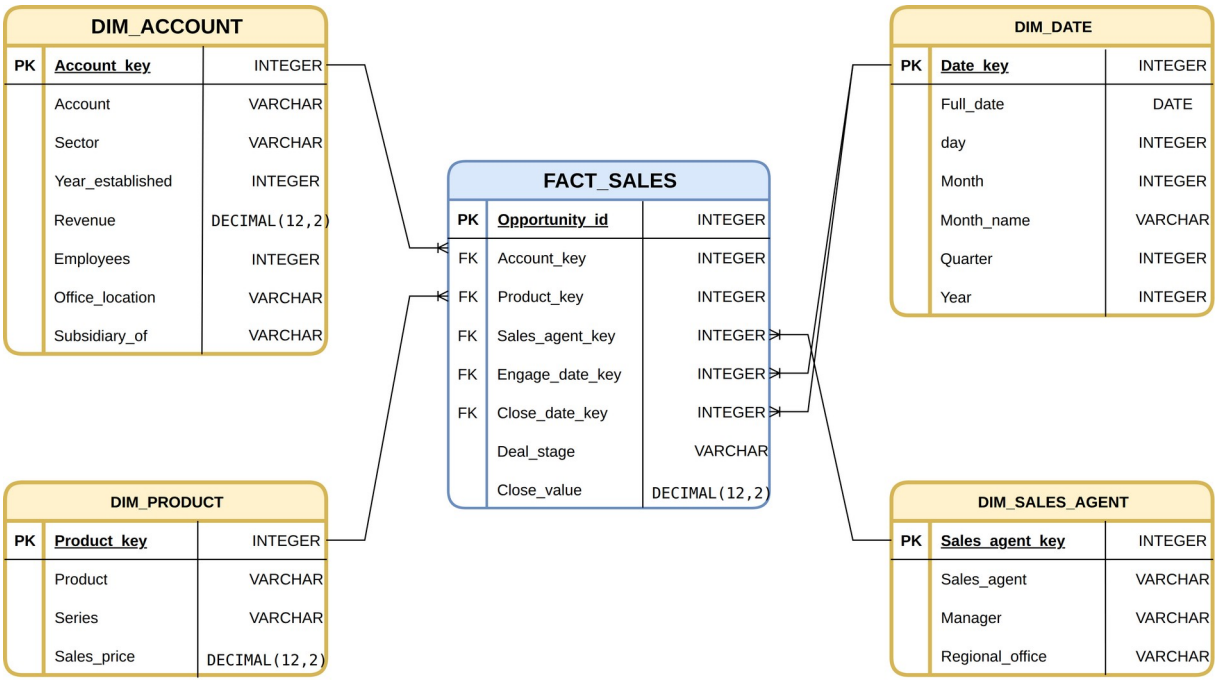


Figure 3: Physical schema with column data types.

4. dbt Transformations

The transformations are defined in dbt Core. A critical design decision in this project was the removal of 'sources.yml' to directly reference the DuckDB raw tables in the staging layer, simplifying the dependency graph.

Example staging model (stg_accounts):

```
with source as (
    select * from raw.accounts
),
renamed as (
    select
        account as account_name,
        sector,
        year_established,
        revenue,
        employees,
        office_location
    from source
)
select * from renamed
```

5. dbt Documentation

dbt Docs provides an interactive data catalog and lineage graph for the project.

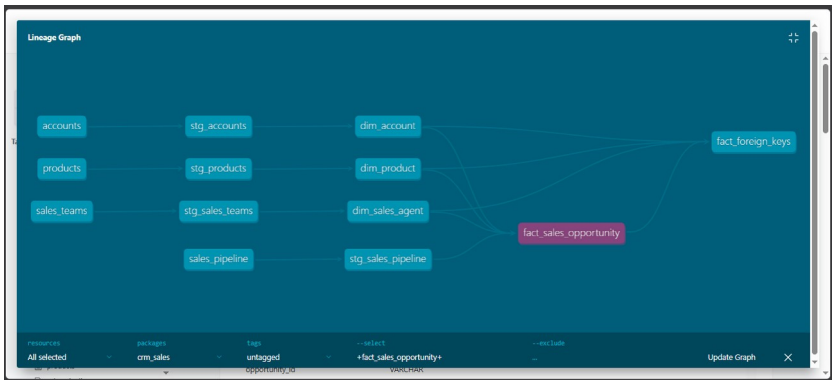


Figure 4: Verified screenshot of the dbt Docs Lineage Graph.

6. Airflow Orchestration

The pipeline is orchestrated using an Apache Airflow DAG named 'crm_sales_pipeline'.

The DAG defines three sequential PythonOperator tasks running the dbtRunner API:

1. **dbt_seed:** Loads configuration or static mapping data.
2. **dbt_run:** Executes all models.
3. **dbt_test:** Executes data quality tests.

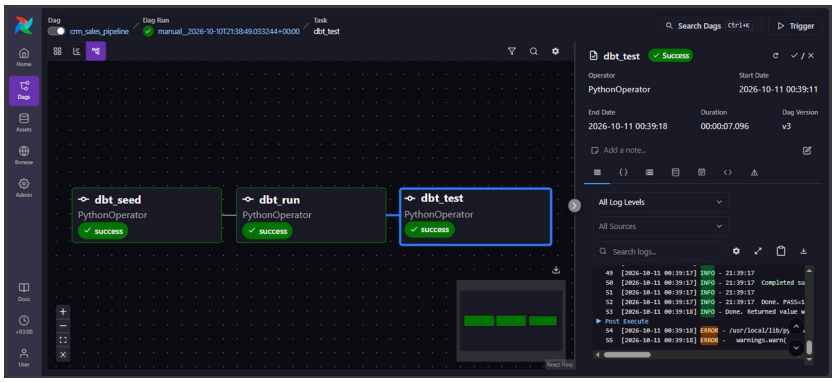


Figure 5: Verified screenshot of the Airflow execution UI.

7. Data Quality and Validation

Data quality is enforced through 17 executed tests:

- **Primary Key tests:** unique and not_null constraints on all staging and dimension keys.
- **Referential Integrity:** fact_foreign_keys validates relationships between the fact table and dimensions.

All tests successfully pass in the verified environment execution.